COURCHAY Arthur - LAUMOND Clément

Repo GitHub : https://github.com/ClementLmd/dev-docker-projet-final

Conteneurisation d'une application Web multi-tiers

Partie 1 — Conception de l'architecture

L'application fournie sépare déjà deux rôles : des pages PHP qui affichent et enregistrent des articles, et une base MySQL qui les conserve. Cette partie fixe les services, le réseau, le volume et les variables qui les feront tourner dans des conteneurs distincts.

1. Application fournie

Le code livré est dans sources/app et sources/db. index.php lit la table articles et affiche les billets du plus récent au plus ancien. La même page contient le formulaire d'ajout (titre, auteur, contenu), envoyé en POST à validation.php. validation.php contrôle les trois champs, prépare un INSERT et renvoie vers l'accueil. La connexion PDO est dans db-config.php : hôte localhost, base test, compte test.

articles.sql crée la base test, le compte test@localhost, puis la table articles.

Colonne	Rôle
id	Clé primaire, entier non signé, auto-incrémenté
title	Titre, 100 caractères au plus
author	Auteur, 100 caractères au plus
date	Horodatage, rempli à l'insertion
content	Texte de l'article

Deux points de ce code empêchent une connexion entre conteneurs. db-config.php fixe l'hôte à localhost : dans le conteneur qui exécute PHP, ce nom désigne le conteneur lui-même. articles.sql crée l'utilisateur test@localhost. Une session ouverte depuis le conteneur web arrive d'une autre adresse du réseau Docker, donc ce compte ne la reçoit pas.

2. Services

La pile compte trois services. Le service web sert les pages. Le service db conserve les articles. Adminer sert à consulter la table pendant les essais ; l'application PHP fonctionne sans lui.

Service	Image	Rôle	Accès depuis le Mac
web	php:8.2-apache	Pages PHP, Apache	127.0.0.1:8080 vers le port 80
db	mysql:8.4	Base articles	Aucun port publié
adminer	adminer	Consultation SQL	127.0.0.1:8081

Le service web utilise PHP 8.2 avec Apache. Les pages sont des fichiers PHP, sans étape de build. Apache publie le DocumentRoot sur le port 80, comme le fait déjà l'application. L'image MySQL est mysql:8.4 : le tag fixe la branche 8.4. Le port 8080 est lié à l'interface locale du Mac. MySQL reste sur le réseau des conteneurs : seuls web et adminer ouvrent le port 3306.

3. Schéma

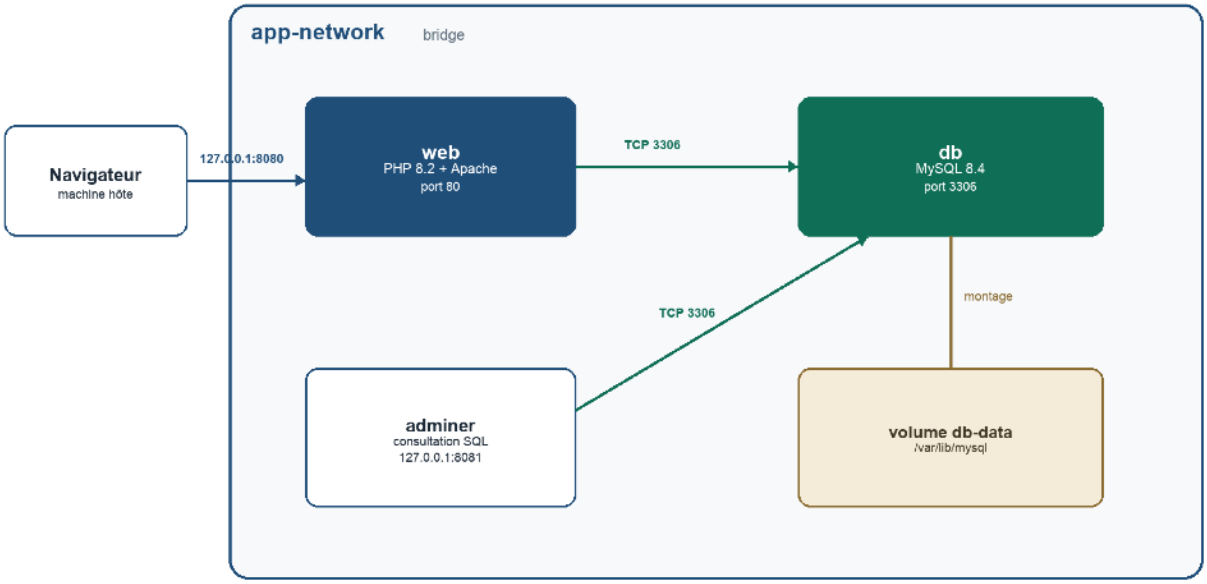


Figure 1. Services, réseau app-network et volume db-data.

4. Réseau

Les trois services partagent un réseau bridge nommé app-network. Le conteneur web joint MySQL par le nom de service db, résolu par le DNS interne de Docker. Aucune adresse IP n'est écrite dans la configuration PHP.

Ce réseau isole la pile des autres conteneurs de la machine. docker network ls le montre à côté du bridge par défaut.

5. Volume

Les fichiers de données MySQL sont dans un volume nommé db-data, monté sur /var/lib/mysql. docker compose down, sans l'option -v, supprime les conteneurs et conserve le volume. Les articles saisis restent présents au relancement.

Le code PHP et le script SQL restent hors de ce volume. Le code est copié dans l'image du service web. Le script d'initialisation est lu au premier démarrage de la base.

6. Variables d'environnement

Les mots de passe et le nom de la base sont dans le fichier .env, ignoré par Git. .env.example est le modèle versionné. Il contient les valeurs de développement local.

Variable	Consommateur	Valeur locale
APP_PORT	Publication du service web	8080
DB_HOST	PHP	db
DB_PORT	PHP	3306
DB_NAME	PHP	test
DB_USER	PHP	test
DB_PASS	PHP	test
MYSQL_DATABASE	Entrypoint MySQL	test

MYSQL_USER	Entrypoint MySQL	test
MYSQL_PASSWORD	Entrypoint MySQL	test
MYSQL_ROOT_PASSWORD	Entrypoint MySQL	root

DB_NAME, DB_USER et DB_PASS restent identiques à MYSQL_DATABASE, MYSQL_USER et MYSQL_PASSWORD. PHP se connecte avec le compte créé par l'image MySQL au premier démarrage.

7. Connexion entre web et db

db-config.php prend l'hôte, la base et le compte dans DB_HOST, DB_NAME, DB_USER et DB_PASS. L'hôte vaut db, le nom du service sur app-network.

L'image MySQL crée la base et le compte applicatif à partir de MYSQL_DATABASE, MYSQL_USER et MYSQL_PASSWORD. Ce compte accepte les connexions des autres conteneurs du réseau. Le script déposé dans /docker-entrypoint-initdb.d/ crée uniquement la table articles.

Partie 2 — Images

Deux images suffisent. Le service web est construit à partir de php:8.2-apache. Le service db reprend mysql:8.4 et y ajoute uniquement le script de création de la table. Les mots de passe restent hors de ces fichiers. Ils sont fournis au lancement, par les variables d'environnement.

1. Service web

Le Dockerfile à la racine produit l'image web. Apache écoute sur le port 80, le processus tourne avec l'utilisateur www-data déjà présent dans l'image Debian (uid 33). Il n'y a pas de second compte à créer.

Instruction	Rôle
FROM php:8.2-apache	Image officielle. PHP et Apache sont déjà assemblés. Le tag fixe la branche 8.2.
apt-get install libcap2-bin	Fournit setcap, utilisé pour autoriser le port 80 sans être root.
docker-php-ext-install pdo_mysql mysqli	Le code ouvre MySQL avec PDO. L'extension mysqli est installée en plus, pour disposer des deux pilotes.
setcap cap_net_bind_service	Pose la capacité sur /usr/sbin/apache2. www-data peut alors ouvrir le port 80.
chown des dossiers Apache	Le fichier PID et les journaux sont écrits dans /var/run/apache2, /var/lock/apache2 et /var/log/apache2. www-data doit en être propriétaire.
apt-get clean et suppression des listes	Les index de paquets ne servent plus une fois les extensions installées. Les retirer allège la couche.
COPY app.conf, a2ensite app	a2dissite retire le site par défaut. a2ensite active app.conf. Le DocumentRoot est /var/www/html. Options -Indexes retire le listing des dossiers. AllowOverride None ignore les .htaccess. Le module PHP est déjà chargé par l'image. rewrite reste désactivé : les adresses sont des fichiers, index.php et validation.php.

COPY --chown sources/app	Le code est copié dans /var/www/html et appartient à www-data. Les droits du dépôt sont gardés.
USER www-data	Le processus du conteneur n'est pas root. La vérification id renvoie uid 33.
EXPOSE 80	Documente le port écouté dans le conteneur. La publication vers le Mac se décide au lancement.
HEALTHCHECK	PHP ouvre une socket vers 127.0.0.1:80. Aucun paquet curl n'est ajouté pour ce contrôle.

Il n'y a pas de compilation d'assets. Un build multi-étapes n'enlèverait aucun outil de construction, parce qu'il n'y en a pas. L'image reste en une seule étape.

L'image a été construite puis lancée. Apache 2.4.69 et PHP 8.2.35 démarrent. getcap confirme cap_net_bind_service sur apache2. La page d'accueil répond en HTTP 200 et le healthcheck passe à healthy. Sans base jointe, le message PDO cite l'hôte db : la configuration ne pointe plus vers localhost. L'image pèse 717 Mo.

2. Service base

docker/mysql/Dockerfile part de mysql:8.4. Il copie articles.sql dans /docker-entrypoint-initdb.d/01-articles.sql. Au premier démarrage, l'entrypoint officiel crée la base et le compte applicatif à partir des variables d'environnement, puis exécute ce script dans la base MYSQL_DATABASE. Le script ne contient plus que le CREATE TABLE de articles.

Choix	Rôle
FROM mysql:8.4	Image officielle. Le tag fixe la branche 8.4.
COPY du script SQL	La table est créée au premier démarrage, seulement si le volume de données est vide.
Pas de variable ENV	Un mot de passe écrit ici resterait lisible dans l'historique de l'image. Au lancement, MYSQL_ROOT_PASSWORD fixe le compte root, MYSQL_DATABASE la base, MYSQL_USER et MYSQL_PASSWORD le compte utilisé par PHP.
Pas d'instruction USER	L'entrypoint prépare les dossiers en root, puis passe à l'utilisateur mysql avec gosu avant de lancer le serveur. Le processus mysqld n'est pas root.

db-config.php lit DB_HOST, DB_PORT, DB_NAME, DB_USER et DB_PASS. Une variable absente arrête la page avec un message explicite, au lieu de retomber sur localhost. L'image MySQL construite pèse 1,1 Go. Le script SQL y est bien présent.

3. Contexte de build et droits

.dockerignore exclut .git, .env, les rapports, la documentation et l'archive du sujet. Ces fichiers n'entrent pas dans le contexte envoyé au démon, et un COPY trop large ne peut pas les embarquer dans une couche.

Aucun chmod 777 n'est utilisé. Ce mode rend le fichier lisible et modifiable par tout utilisateur du conteneur et, selon le montage, de l'hôte. Un processus qui peut écrire dans /var/www/html peut remplacer index.php. L'application n'enregistre pas de fichier : Apache a besoin de lire le code, pas de le modifier. Les fichiers restent ceux copiés, appartenant à www-data.

Partie 3 — Orchestration

docker-compose.yml lance les trois services du projet articles. Les mots de passe ne sont pas écrits dans ce fichier : Compose les lit dans .env, qui n'est pas versionné. .env.example en est le modèle.

1. Sections du fichier

Section	Rôle
name	Nomme le projet articles. Les conteneurs s'appellent web, db et adminer.
web.build	Construit le Dockerfile à la racine. L'image produite s'appelle articles-web:dev.
web.ports	Publie le port 80 du conteneur sur 127.0.0.1 et le port APP_PORT, 8080 par défaut. Les autres interfaces du Mac ne reçoivent pas ce port.
web.environment	Fournit DB_HOST, DB_PORT, DB_NAME, DB_USER et DB_PASS. DB_PASS est obligatoire : le lancement s'arrête si .env ne la définit pas.
web.depends_on	Démarre web une fois db healthy. Un conteneur MySQL démarré n'est pas encore prêt à accepter une connexion.
web.healthcheck	PHP ouvre une socket vers 127.0.0.1:80, comme dans le Dockerfile.
web.restart	unless-stopped relance le conteneur s'il s'arrête, sauf arrêt demandé.
db.build	Construit docker/mysql/Dockerfile. L'image produite s'appelle articles-db:dev.
db.environment	Passe MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER et MYSQL_PASSWORD. Les deux mots de passe viennent de .env.
db.volumes	Monte le volume nommé db-data sur /var/lib/mysql.
db.healthcheck	mysqladmin ping interroge le serveur local. Le start_period de 40 s laisse finir la première initialisation.
db, sans ports	Aucun port MySQL n'est publié sur le Mac. Seuls les conteneurs d'app-network joignent le port 3306.
adminer	Image adminer:4.8.1. Le port 8080 du conteneur est publié sur 127.0.0.1:8081. Le serveur proposé à la connexion est db.
networks	Crée le réseau bridge nommé app-network et y place les trois services.
volumes	Déclare le volume nommé db-data. docker compose down, sans -v, ne le supprime pas.

2. Lancement et persistance

docker compose up -d --wait a attendu l'état healthy des trois conteneurs. La page http://127.0.0.1:8080/ répond en HTTP 200, sans erreur PDO. Adminer répond en HTTP 200 sur http://127.0.0.1:8081/.

Un article « Essai de persistance » a été ajouté par le formulaire. docker compose down a supprimé les conteneurs et le réseau app-network. Le volume db-data était encore listé. Après un nouveau docker compose up -d --wait, le même article est affiché sur la page d'accueil.

Partie 4 – Réseaux

La pile applicative repose sur le réseau bridge nommé app-network. Les conteneurs communiquent entre eux par leurs noms de service sans exposer inutilement de ports sur la machine hôte.

1. Topologie et inspection du réseau

docker network ls liste app-network avec le pilote bridge et un périmètre local. La commande docker network inspect app-network détaille le sous-réseau IPv4 alloué (172.26.0.0/16), la passerelle (172.26.0.1) et les adresses attribuées à chaque conteneur au démarrage.

Conteneur	Adresse IP	Ports internes	Publication hôte
db	172.26.0.2	3306/tcp, 33060/tcp	Aucun (interne au réseau)
web	172.26.0.3	80/tcp	127.0.0.1:8080 vers le port 80
adminer	172.26.0.4	8080/tcp	127.0.0.1:8081 vers le port 8080

2. Résolution DNS et connectivité inter-conteneurs

Sur les réseaux bridge personnalisés, Docker intègre un résolveur DNS interne (127.0.0.11). Chaque conteneur résout dynamiquement les autres par leur nom de service Compose, sans recourir à des adresses IP fixes. La commande getent hosts db exécutée dans web confirme la résolution de db en 172.26.0.2. C'est ce mécanisme qui permet à db-config.php de joindre MySQL avec DB_HOST=db.

Deux tests complémentaires valident la liaison entre web et db :

Test	Commande exécutée	Résultat et interprétation
Connectivité niveau 3 (ICMP)	docker exec web ping -c 3 db	3 paquets transmis et reçus sans aucune perte, avec une latence moyenne de 0,5 ms. Le paquet iputils-ping a été ajouté au Dockerfile web pour autoriser ce diagnostic sans alourdir l'image (191 Ko).
Connectivité niveau 4/7 (TCP)	docker exec web curl -v http://db:3306	curl résout db en 172.26.0.2 et ouvre la connexion TCP sur le port 3306 (Connected to db (172.26.0.2) port 3306). Le message Received HTTP/0.9 when not allowed provient du handshake binaire émis par MySQL en réponse à une requête HTTP, confirmant que le démon écoute et répond.

3. Isolation et intérêt du bridge personnalisé

L'utilisation d'un réseau bridge utilisateur apporte trois garanties essentielles par rapport au bridge par défaut de Docker :

Propriété	Bénéfice technique et sécurité
Résolution DNS native	Les conteneurs sur app-network s'adressent par nom de service. Le réseau bridge par défaut impose des liaisons manuelles (--link, dépréciées) ou des adresses IP fixes instables.

Étanchéité et cloisonnement	Les conteneurs lancés hors de ce Compose ou sur le bridge par défaut ne peuvent ni voir ni interroger db. Une compromission sur un conteneur tiers ne donne aucun accès réseau à cette base.
Protection du port 3306	Aucun port MySQL n'est publié sur la machine hôte. Seuls web et adminer, placés sur app-network, peuvent envoyer des requêtes vers 3306. La surface d'exposition de l'hôte reste limitée à 8080 et 8081.

Partie 5 – Sécurité

La sécurité de la pile est appliquée selon le principe de défense en profondeur : audit des vulnérabilités statiques, étanchéité de la gestion des secrets, restriction radicale des privilèges à l'exécution et interdiction stricte de permissions permissives.

1. Scans de vulnérabilités (Trivy)

Les deux images produites ont été soumises à un scan statique de vulnérabilités avec Trivy (aquasec/trivy) ciblant les sévérités HIGH et CRITICAL.

Image scannée	Base OS	Vulnérabilités détectées	Failles critiques et analyse
articles-web:dev	Debian 13 (Trixie)	120 (117 HIGH, 3 CRITICAL)	libxml2 (CVE-2026-6653) et linux-libc-dev (CVE-2026-93221). Ces failles sont héritées de l'image de base Debian. Pour une mise en production, une base Alpine réduirait drastiquement ce volume.
articles-db:dev	Oracle Linux / MySQL 8.4	35 (33 HIGH, 2 CRITICAL)	certifi (GHSA-6q48-6c7w-9gw2) et gosu (CVE-2025-68121). L'image officielle isole le processus mysqld avec l'utilisateur mysql après initialisation.

2. Gestion des secrets

Aucun mot de passe ni identifiant sensible n'est consigné dans les Dockerfiles ou le fichier docker-compose.yml. Les secrets sont centralisés dans le fichier .env, qui est exclu du versionnement par .gitignore. Le fichier .dockerignore garantit également qu'un COPY large n'embarque jamais le fichier .env dans une image.

Dans docker-compose.yml, l'interpolation obligatoire (${DB_PASS:?Definir DB_PASS dans .env} et ${MYSQL_ROOT_PASSWORD:?...}) bloque immédiatement le démarrage de la pile si les secrets ne sont pas définis.

3. Limitation des privilèges et durcissement à l'exécution

Le service web a fait l'objet d'un durcissement poussé dans docker-compose.yml afin de confiner strictement le conteneur en cas d'exploitation d'une faille applicative PHP :

Paramètre de sécurité	Mécanisme et justification
USER www-data (UID 33)	Le conteneur web n'exécute aucun processus en root. La compromission d'Apache ne donne aucun accès root sur l'hôte.
read_only: true	Le système de fichiers racine du conteneur est monté en lecture seule. Un attaquant ne peut ni modifier index.php ni déposer de webshell. Une commande touch /var/www/html/test.txt renvoie l'erreur 'Read-only file system'.
security_opt: no-new-privileges	Interdit formellement aux processus du conteneur d'acquérir de nouveaux privilèges via des binaires setuid/setgid.
cap_drop: ALL / cap_add: NET_BIND_SERVICE	Toutes les Linux Capabilities sont retirées. Seule NET_BIND_SERVICE est accordée pour qu'Apache (non-root) puisse ouvrir le port 80.
tmpfs (volumes volatils en RAM)	Des montages temporaires en RAM avec les options noexec,nosuid sont alloués à /var/run/apache2, /var/lock/apache2, /var/log/apache2 et /tmp pour permettre à Apache de consigner son PID et ses logs sans compromettre l'immuabilité du système de fichiers.

4. Bannissement du mode chmod 777

L'utilisation d'un chmod 777 constitue une faille de sécurité majeure car elle octroie les droits de lecture, écriture et exécution à tous les utilisateurs du système. Un attaquant exploitant une vulnérabilité pourrait écraser le code source ou y insérer des portes dérobées.

L'application ne gérant aucun téléversement de fichier, les sources appartiennent à www-data:www-data avec des permissions de lecture stricte (fichiers 644, répertoires 755), protégeant l'intégrité du code.

Partie 6 – Optimisation

L'optimisation des conteneurs réduit l'empreinte disque, accélère les temps de transfert réseau et diminue la surface d'attaque en écartant tout fichier ou paquet superflu.

1. Comparatif de taille des images

Les mesures effectuées sur les images locales et sur des alternatives légères mettent en évidence les gains d'optimisation potentiels :

Service	Image actuelle	Variante allégée	Gain d'espace mesuré
Service Web (PHP/ Apache)	articles-web:dev (711 Mo)	php:8.2-fpm-alpine (118 Mo)	- 83,4 % d'empreinte disque (gain de 593 Mo). Alpine élimine tout l'espace utilisateur Debian.

Service Base (SQL)	articles-db:dev (1,1 Go)	mariadb:lts (460 Mo)	- 58,2 % d'empreinte disque (gain de 640 Mo). MariaDB allège le runtime par rapport à MySQL 8.4.
Contexte de build Docker	Contexte brut (~3,5 Mo)	Contexte épuré (6,65 Ko)	- 99,8 % de données transférées au démon lors du docker build grâce au .dockerignore.

2. Nettoyage des couches (Layers)

Dans le Dockerfile du service web, la création de couches intermédiaires a été strictement contenue :

- **Chaînage des commandes :** L'ensemble des opérations apt, l'installation des extensions PHP, la pose de la capacité cap_net_bind_service et le nettoyage sont regroupés dans un unique RUN relié par des &&. Cela produit une seule couche applicative de 2,02 Mo au-dessus de l'image de base.
- **Option --no-install-recommends :** Bloque l'installation des dépendances suggérées et des documentations superflues.
- **Suppression immédiate du cache apt :** apt-get clean et rm -rf /var/lib/apt/lists/* sont exécutés dans la même instruction pour ne laisser aucun fichier temporaire dans la couche finale.

3. Analyse et mise en œuvre du multi-stage build

L'application fournie est constituée de scripts PHP interprétés sans phase de compilation d'assets (aucun compilateur Sass, bundler Webpack ou dépendance Composer externe). Un multi-stage build visant à exclure des outils de build Node ou C++ n'est donc pas requis en fonctionnement nominal.

Toutefois, pour répondre aux exigences avancées d'intégration continue et de qualité, nous avons créé et testé le fichier Dockerfile.multistage. Ce fichier décompose la chaîne en deux étapes distinctes :

- **Étape 1 (builder sous php:8.2-cli-alpine) :** Effectue un contrôle qualité automatique par linting syntaxique (php -l) de tous les fichiers du projet avant toute publication.
- **Étape 2 (runner sous php:8.2-apache) :** Ne copie que les sources validées (COPY --from=builder) dans le conteneur final sans aucun résidu de test.

4. Efficacité du fichier .dockerignore

Le fichier .dockerignore a été étendu pour exclure l'historique .git, les secrets .env, les rapports Word/Pages, la documentation Markdown, les archives zip/rar et les fichiers d'environnement Compose. Le contexte de build envoyé au démon lors d'un docker build est ainsi réduit à 6,65 Ko, garantissant un build quasi instantané et une sécurité accrue.

Partie 7 – CI/CD et publication

L'automatisation du cycle de vie logiciel est assurée par un pipeline d'intégration et de déploiement continus (CI/CD) déployé sous GitHub Actions (.github/workflows/docker.yml). Chaque push ou pull request sur la branche main déclenche automatiquement la validation du code, la construction des conteneurs, un scan de vulnérabilités et la publication sur Docker Hub.

1. Architecture du workflow GitHub Actions

Le pipeline est structuré en deux jobs séquentiels afin d'éviter toute construction inutile d'images en cas d'erreur de programmation :

Job	Objectif technique	Actions et commandes exécutées
lint-and-test	Validation statique et syntaxique pré-build	Vérifie la syntaxe de tous les fichiers PHP du projet avec 'php -l' et valide la conformité du fichier docker-compose.yml via 'docker compose config'.
build-and-publish	Construction, audit et publication	Configure Docker Buildx avec cache GitHub Actions, extrait les métadonnées de commit, construit les images web et db, exécute un scan de vulnérabilités Trivy et pousse les images sur Docker Hub.

2. Stratégie d'étiquetage (Tags Docker Hub)

Les images publiées sur Docker Hub reçoivent systématiquement un double étiquetage afin d'allier commodité d'utilisation et traçabilité stricte :

- **Tag 'latest' :** Désigne la version de référence déployée depuis la branche principale main.
- **Tag 'sha-<commit>' :** Associe de façon immuable l'image Docker à l'empreinte cryptographique exacte du commit Git, permettant un rollback immédiat en cas de régression.

3. Sécurisation des accès et gestion des secrets

L'authentification auprès de Docker Hub ne fait intervenir aucun mot de passe en clair. Deux secrets chiffrés sont enregistrés dans les paramètres du dépôt GitHub :

Secret GitHub	Rôle	Mesure de protection
DOCKERHUB_USERNAME	Identifiant du compte Docker Hub	Chiffré par l'API GitHub et masqué automatiquement dans les logs de build.
DOCKERHUB_TOKEN	Jeton d'accès personnel (PAT - Personal Access Token)	Généré sur Docker Hub avec des permissions restreintes (Read & Write) sans donner accès aux fonctions d'administration du compte.

De plus, l'opération de push est conditionnée : si le pipeline s'exécute lors d'une pull request externe ou sans secrets renseignés, les étapes de test et de build sont validées sans générer d'erreur de publication.

Partie 8 — Tests et documentation

Le README.md décrit l'architecture, les prérequis, les commandes de déploiement, les variables d'environnement et la procédure de test.

Les images sont publiques sur Docker Hub, avec les tags latest et sha-<commit> :

clemlmd/articles-web — https://hub.docker.com/r/clemlmd/articles-web

clemlmd/articles-db — https://hub.docker.com/r/clemlmd/articles-db

Le dépôt a été cloné dans un dossier neuf, puis lancé avec docker compose up -d --build --wait. Les services web, db et adminer sont passés à healthy. La page http://127.0.0.1:8080/ a répondu en HTTP 200.